

**Analog Audio Amplifier With Tone Control, ESP32
MP3 Decoding, and Ultrasonic Volume Control**

Course Name: Analog Circuits

Course Code: BECE206L

Slot: D1+TD1

Class Number: VL2025260102660

Faculty Name: Dr. SANDHANA MAHALINGAM M

A course based design project by:

Saksham Arora: 24BEC0185

Daspati Anuraag Prahas: 24BEC0547

Srajan Gupta: 24BEC0066

Surya Pratap Singh: 24BML0069

Table of Contents

1. Introduction
2. Acknowledgement
3. Abstract
4. Literature Survey
5. Hardware Components Review
6. Project Block Diagram
7. Workflow Explanation
8. Software (ESP32 Code)
9. MATLAB Signal Analysis
10. Hardware Implementation
11. Circuit Diagrams
12. Future Work
13. Conclusion
14. References

Acknowledgement

I would like to express my sincere gratitude to Dr. Sandhana Mahalingam M, School of Electronics Engineering (SENSE), VIT University, for his constant guidance, insightful feedback, and technical support throughout the development of my audio amplifier project. His valuable suggestions and encouragement greatly contributed to the successful completion of this work.

I would also like to thank the faculty members and lab staff of the School of Electronics Engineering for their assistance and for providing access to the necessary equipment and components required for the design, simulation, and testing of the amplifier.

My sincere thanks to the Chancellor of VIT, Dr. G. Vishwanathan, the Vice Presidents, and the Vice Chancellor for creating an environment that promotes hands-on learning and innovation, and for providing the infrastructural facilities essential for completing this project.

Finally, I appreciate my fellow teammates and peers for their collaboration, discussions, and constructive feedback, which helped refine and enhance the performance and presentation of this work.

Abstract

This technical report presents the design, implementation, simulation, and testing of a modular analog audio amplifier system that combines modern embedded audio playback with classic analog tone shaping and a discrete Class-AB power stage. The system uses an ESP32 microcontroller to decode MP3 files from a FAT32 microSD card and stream PCM audio (I2S or internal DAC) to an external NE5532-based tone control module. The tone board performs Baxandall-style bass/treble shaping, whose output is buffered by a BC109BP small-signal stage, biased by a compact PR1 + 1N4148 $\times 2$ + 2N3906 network, and amplified by a complementary BD139/BD140 push-pull Class-AB output pair with emitter resistors for current sharing. Two HC-SR04 ultrasonic sensors provide touchless human-machine interaction: one continuously maps hand distance to discrete volume levels (via a digital potentiometer or channel selector) and the other toggles speaker mute through a relay. The report documents firmware for the ESP32, wiring and bench-test procedures, Multisim circuit simulations of the power stage, and MATLAB scripts used for frequency-domain analysis and filter verification. Simulation and practical tests demonstrate stable biasing, effective tone shaping, reliable ultrasonic control, and clean audio drive to an 8 Ω load with appropriate heatsinking and supply decoupling. The conclusion summarizes performance, limitations, and recommended improvements—such as an external I2S DAC for higher fidelity, enhanced gesture robustness, and optional Bluetooth/streaming integration—providing a practical roadmap for future development and optimization.

Introduction

Audio Amplifiers

Analog audio amplification remains a cornerstone of audio electronics, with a rich history of design trade-offs between distortion and efficiency. Amplifiers are classified (Class A, B, AB, D, etc.) by how their output devices are biased and driven. For example, Class A amplifiers conduct over the full 360° of the waveform, yielding excellent linearity and minimal distortion, but they waste a lot of power (only $\sim 25\text{--}50\%$ theoretical efficiency) as heat. Class B push–pull amplifiers conduct only half the cycle (180° per device), greatly improving efficiency (up to $\sim 78.5\%$) but introducing crossover distortion at the waveform junction. Class AB biases both devices slightly on so that each conducts more than 180° , which greatly reduces crossover distortion while retaining higher efficiency than Class A. Class D amplifiers switch devices fully on/off (using PWM or sigma-delta modulation) to achieve very high efficiencies ($>80\text{--}90\%$), at the cost of requiring output filtering and digital control. In consumer audio, Class AB is especially common as a practical compromise – it provides good fidelity with moderate efficiency and cost, which is why this project adopts a Class AB output stage.

- Class A: 100% conduction (360°); highest linearity and lowest crossover distortion, but very low efficiency ($\sim 25\text{--}50\%$). The output devices always draw current, generating much heat.
- Class B: 50% conduction (180° per device); improved efficiency (max $\sim 78.5\%$), but introduces crossover distortion

at the waveform join. Two devices in push–pull share the waveform.

- Class AB: >50% conduction (each device slightly on during the other's half-cycle); a compromise offering low distortion (almost as good as Class A for small signals) with better efficiency than pure Class A. Biasing is chosen to minimize the “dead zone.”
- Class D: Devices act as high-frequency switches, using PWM or delta-sigma modulation, with output averaged by a low-pass filter; this yields >80–90% efficiency. High-quality Class D designs can rival analog amps in sound, and their small size makes them popular in modern mobile/home audio.

These trade-offs explain the choice of a Class AB amplifier for this project: it balances sound quality and efficiency in a simple analog design suitable for a home audio system.

Tone Control in Audio Systems

Tone control circuits allow listeners to tailor the bass and treble balance of playback. In high-fidelity and consumer audio systems, tone knobs boost or cut specific frequency bands to suit personal preference or room acoustics. For example, a listener might add bass to compensate for weak speakers or raise treble for clarity. Passive tone control networks (using only resistors, capacitors, and inductors) are the simplest approach: they attenuate (cut) certain frequencies by filtering, but they cannot increase (boost) any band. In contrast, active tone controls use an operational amplifier and feedback to amplify and equalize. This allows both cut and boost of bass/treble without excessive loss of signal level.

- Passive tone controls: Built from R-C (or R-L-C) networks with no power supply. They can attenuate low or high bands (“bass cut” or “treble cut”) by exploiting reactance, but they cannot amplify. Such circuits were common in vintage radios and guitar amps.
- Active tone controls: Use op-amps (or vacuum tubes) so that the network forms part of a feedback loop. This enables true bass and treble boost as well as cut, while maintaining flat response at center settings. The famous *Baxandall* tone control (invented 1952) is an analog example: it provides independent bass and treble shelves that can each be swung +/- (typically ± 10 – 15 dB).

The Baxandall circuit cleverly uses negative feedback around an amplifier stage to compensate for the ~ 20 dB attenuation of the passive network. As a result, with its controls at midpoint the frequency response is flat, and turning the knobs gives smooth bass or treble shelving. Its shelving filters have wide “Q” so the transitions are gradual and musical. Modern amplifiers and mixers still use Baxandall-style (or similar) circuits precisely because they sound transparent. Many hi-fi receivers and instrument amplifiers use op-amp-based tone modules (today avoiding large inductors). In short, tone controls let users shape audio timbre in music playback and recording, adjusting for speaker/room characteristics or personal taste.

Evolution of Embedded Audio Systems

Audio playback technology has steadily moved from analog hardware to digital systems. In earlier decades, music listening relied on analog media (turntables, cassettes, FM radio) that used dedicated analog circuitry for tuning and amplification. Over

time, digital formats took over: by the 1990s the MP3 codec and portable players revolutionized how audio was stored and transmitted. Today's embedded audio systems typically use microcontrollers or SoCs to decode compressed audio (MP3, WAV, AAC) from storage. MP3 is especially common in embedded designs because its lossy compression makes files much smaller than raw PCM or WAV.

Embedded systems may decode audio in hardware (e.g. dedicated MP3 decoder chips) or entirely in software. Hardware solutions like the VS1053B allow an MCU to stream MP3 bytes to the chip, which outputs analog audio. Alternatively, a modern microcontroller with sufficient CPU (e.g. ARM Cortex-M4/M7 or the ESP32) can run MP3 decoder libraries in firmware. The Digi-Key application note, notes that selecting an MCU “that supports audio encoding/decoding” is key for efficient implementation. In practice, many microcontrollers (including the ESP32) have become powerful enough to decode MP3 on-the-fly using DMA and libraries, while handling file I/O from SD cards or USB.

This evolution means that a microcontroller board can now act as a full-featured audio player. For example, one hackathon project built a retro “tape” player using an ESP32 WROOM: it read MP3 files off an SD card and drove an I2S audio amp (MAX98357A) to power a speaker. The designer noted that the “ESP32 has more than enough computational horsepower to chew through MP3 files” from SD storage. Similarly, internet radio and voice-changer projects demonstrate that an ESP32 can handle streaming or audio processing tasks. Thus, modern embedded audio marries old and new: a digital core for convenience and connectivity, but still an analog stage for final amplification. This

project follows that model by having the ESP32 decode MP3/WAV from its SD card and output through the analog amplifier.

The ESP32 Microcontroller as an Audio Platform

The ESP32 is a powerful multimedia microcontroller that integrates Wi-Fi/Bluetooth with versatile peripherals. Crucially for audio, it provides an I2S peripheral for high-quality digital audio streaming. An ESP32 can send PCM data over I2S to an external DAC or Class D amplifier chip, allowing easy connection to speakers. In fact, many ESP32 audio projects use I2S amplifiers (such as the Texas Instruments PCM5122 or the small MAX98357A modules) for line-level output. The ESP32 also includes two built-in 8-bit DAC channels (typically on GPIO25 and GPIO26), though these are low-resolution (phone-quality only) and not suitable for high-fidelity stereo. Consequently, high-end audio use usually relies on external I2S DACs or codecs for better performance.

Besides audio I/O, the ESP32 supports SD cards (via SPI or SDMMC interfaces) so it can store and read large audio libraries. It has plenty of flash/RAM for code and buffers. Its dual cores (up to 240 MHz) are fast enough to decode compressed formats: as noted, projects have successfully built standalone MP3 players on ESP32 without extra hardware. The ESP32's Wi-Fi/BT connectivity also enables streaming and networked audio features (e.g. internet radio). In short, the ESP32's combination of I2S support, I/O, processing power, and file system interfaces makes it well-suited as the heart of a digital audio system.

Human-Computer Interaction and Touchless Control

Modern embedded systems increasingly incorporate intuitive human-computer interfaces. Beyond traditional knobs and buttons, designers use gesture and proximity sensors to create natural user experiences. Ultrasonic sensors are one such interface: by emitting high-frequency sound and measuring echo time, modules like the HC-SR04 can detect hand distance or motion without touch. This enables *touchless* controls – for example, moving a hand nearer the sensor could raise volume. Research and industry have demonstrated advanced ultrasonic gesture interfaces: for instance, Ultrahaptics (now UltraLeap) combines a Leap Motion hand tracker with an array of ultrasonic transducers to create mid-air “buttons” and sliders. At CES, Ultrahaptics showed prototypes where users could adjust a stove or stereo by waving hands; one demo embedded an ultrasonic interface in a speaker system to control volume and track selection purely by air gestures.

In embedded projects, even simple ultrasonic modules can achieve basic gesture control. For our amplifier, using an ultrasonic range sensor for volume means the user can increase or decrease sound level by sliding a hand closer or farther—no physical contact needed. This aligns with a broader trend toward touchless HCI for convenience and hygiene. In summary, intuitive gesture interfaces (like ultrasonic volume control) bring a modern, futuristic feel to what might otherwise be a classic analog audio device.

Project Relevance and Integration

This project weaves together legacy analog design and contemporary digital technology. The analog Class-AB amplifier and Baxandall tone control deliver the warm, responsive sound shaping valued in traditional hi-fi equipment. At the same time, the ESP32 and SD-based MP3 decoder bring the convenience of digital media and connectivity that consumers expect today. The ultrasonic volume control adds an intuitive user interface, illustrating how human-computer interaction can enhance an embedded system.

Such integration is justified both practically and educationally. For home audio, this design could serve as a flexible stereo system: it can play files or streams like a modern player, yet still offer physical bass/treble adjustments for listening preference. As an educational project, it teaches analog filter and amplifier design alongside microcontroller programming and HCI concepts. Blending analog and digital elements also exemplifies how engineering evolves: analog tone circuits are not obsolete but can complement digital processing.

By combining these threads, the project highlights the versatility of embedded platforms like the ESP32. The result is a system where vintage-style tone controls and a novel gesture sensor coexist with digital music playback. In doing so, it addresses use cases from home audio enjoyment to hands-on learning, demonstrating that old and new audio technologies can be harmoniously integrated in a single device.

Literature Survey

1. Overview and scope

This literature review covers four core technical areas that underpin the project: Class-AB audio amplifier design, analog tone control (Baxandall and variants), embedded MP3/audio playback on microcontrollers (ESP32 and common audio stacks), and ultrasonic sensing for touchless control. For each area I summarize important findings, highlight practical design considerations, and point to authoritative sources (academic papers, datasheets, authoritative application notes, and well-maintained open-source implementations).

2. Class-AB amplifier design and practical considerations

Class-AB output stages are the most common discrete amplifier topology for hobbyist and many consumer hi-fi applications because they balance linearity and efficiency: biasing reduces crossover distortion (relative to Class B) while avoiding the heavy thermal penalty of Class A. Designers must carefully implement bias networks, driver/VAS stages, emitter resistors, and thermal compensation (V_{be} multiplier or diode string) to achieve low THD across load and temperature conditions.

Important findings from the literature

- Reviews and design papers stress that Class-AB biasing and local feedback/topology strongly influence crossover distortion, stability and THD; techniques such as composite devices, VAS current-mirrors, and proper local decoupling

are widely recommended to reduce distortion and improve reliability. For a recent consolidated discussion of modern Class-AB implementations and efficiency tradeoffs see Tam (Design of CMOS Class AB Bridged Audio Amplifier) and comparative studies on Class AB vs Class D.

[ScienceDirect+1](#)

- Practical hobbyist/engineering tutorials and design notes provide accessible circuit references (bias network values, emitter resistor sizing, thermal mounting). These are useful for translating academic recommendations into working Multisim simulations and PCB builds. An example review aimed at practical construction and measurement is available in a recent IJCRT preprint and other community articles.

[IJCRT+1](#)

Design takeaways for this project

- Use a V_{be} multiplier or adjustable bias string for thermal tracking of output transistors.
- Include modest emitter resistors (to aid current sharing) and a robust heatsink for power devices.
- Simulate the amplifier with realistic transistor models and include power-supply decoupling and protection elements (short-circuit, thermal).

(Representative publications and tutorials cited above provide formulas and example component values that can be used directly in Multisim.) [ScienceDirect+1](#)

3. Tone control: Baxandall topologies and active implementations

Background

The Baxandall tone control (passive and active variants) is the de-facto standard for consumer preamplifiers where linear, musical boost/cut for bass and treble is required. Its active implementation around an op-amp enables symmetric boost and cut with near-flat response at the mid (neutral) setting.

Authoritative sources & analysis

- The original Baxandall publication and later detailed analyses present exact transfer functions and design equations for shelving networks, along with the practical impact of component tolerances on shelf depth and knee frequency. The canonical articles and annotated reproductions of the Wireless World piece and analytical PDFs are commonly used as design references.
[hifisonix.com+1](#)
- Circuit analysis documents (worked examples) show how the choice of R and C values sets the knee frequencies and the available boost/cut range; active (op-amp) versions restore gain lost in passive networks so that centre positions remain unity gain. These analyses are valuable for choosing component values for desired bass/treble corner frequencies.
[guitarscience.net+1](#)

Practical note for this project

- The NE5532 (dual low-noise op-amp) is an appropriate choice for an active preamp/tone stage because of its low input-referred noise and strong output drive for audio preamp tasks. Use the NE5532 datasheet to confirm supply rails, slew-rate and output swing constraints when choosing component values and expected headroom. [ti.com+1](#)

4. Embedded MP3/audio playback on ESP32 and related libraries

Capabilities of ESP32 for audio

The ESP32 provides I2S hardware for streaming PCM samples to external DACs and supports SD-card interfaces for storing audio files. Its dual cores and available community libraries make real-time MP3 decoding and I2S streaming feasible for high-quality audio projects.

Key implementations and software resources

- The **ESP32-audioI2S** project on GitHub is a widely used, actively maintained library that decodes MP3/AAC/others (Helix/FAAD decoders) and outputs through I2S to external DAC/codec modules (e.g., MAX98357A). It serves as a practical implementation reference for MP3 decoding, buffer management, SD access, and I2S configuration. [GitHub+1](#)
- Tutorials and community writeups (DroneBot Workshop, Random Nerds, and similar) provide pragmatic instructions for wiring I2S peripherals, initializing SD card access, selecting sample rates, and handling common pitfalls (bitclock and LRCLK pin selection, DMA buffer sizing).

These resources help convert library reference code into robust firmware for the ESP32. [DroneBot Workshop+1](#)

Design implications for this project

- Prefer I2S output to a DAC (external or I2S DAC module) rather than the internal 8-bit DAC pins if high fidelity is desired.
- Use a tested library (e.g., ESP32-audioI2S) for MP3 decoding; it provides practical examples for SD filesystem access and playback control, which you can adapt to add volume control hooks or mute control tied to ultrasonic sensors. [GitHub+1](#)

5. Ultrasonic sensing for touchless control (HC-SR04 and gesture mapping)

Sensor characteristics

HC-SR04 ultrasonic modules provide inexpensive proximity/distance sensing (typical range 2–400 cm, ~3 mm resolution under ideal conditions). They operate by a 10 μ s trigger pulse and returning a pulse whose width corresponds to the echo round-trip time; converting pulse width to distance uses the speed of sound. Datasheets and trusted tutorials define timing, voltage requirements (module expects 5 V), and guidance on trigger/echo sequencing. [cdn.sparkfun.com+1](#)

Applications to gesture/volume control

Tutorials and maker projects demonstrate mapping sensor distance to volume levels or to toggle relays for

mute/unmute. The common approach is to filter or debounce distance readings and to map the physical range to a volume scale (often linear mapping for simplicity or logarithmic mapping for perceptual volume). [How To Mechatronics+1](#)

Practical cautions

HC-SR04's Echo output is a 5 V logic pulse; when connecting to a 3.3 V microcontroller (ESP32), use a level shifter or voltage divider to avoid I/O damage. Also account for spurious reflections in real environments (add median filtering or moving average to the measured distances). [cdn.sparkfun.com+1](#)

6. Supporting components: op-amp, digital potentiometer, and codec options

NE5532 op-amp (tone control)

- The NE5532 datasheet documents low input-referred voltage noise, high output drive, and recommended supply rails; it has been a standard audio op-amp for decades. Use the datasheet to size coupling caps, set DC bias for single-ended inputs (if applicable), and ensure the op-amp operates within its linear range for the chosen rail voltages. [ti.com+1](#)

7. Synthesis: how the surveyed work applies to this project

1. **Amplifier topology and simulation:** Use Class-AB topology with biasing and thermal compensation guidelines from the Class-AB literature to set up Multisim simulations

and to predict THD and output power. The academic reviews and applied design notes will guide bias voltage, output device selection, and thermal management.

2. **Tone control design:** Implement an active Baxandall network using the NE5532 op-amp. Apply the design equations from Baxandall analyses to pick RC values for desired knee frequencies and shelf slopes; verify the frequency response with MATLAB (freqz) or SPICE.
3. **MP3 playback stack:** Base the firmware on a tested library such as ESP32-audioI2S for MP3 decoding and I2S output. Route the I2S/DAC output to the tone preamp input, and provide hooks for volume/mute control (via digital potentiometer and relay). The GitHub project provides code, buffer management examples, and decoder configuration that can be adapted. [GitHub+1](#)
4. **Ultrasonic interface:** Use HC-SR04 modules for mute and volume gestures, with filtering and level-shifting for safe ESP32 interfacing. Mapping strategies (linear or logarithmic) and debounce filtering are well documented in practical tutorials. [cdn.sparkfun.com+1](#)
5. **Component selection:** Use NE5532 for tone control (datasheet parameters), MCP41xxx/MCP42xxx series digital pots for MCU-controlled attenuation, and optionally VS1053 family if offloading decoding is desired. Consult datasheets for exact electrical constraints when wiring the analog chain. [ti.com+2Microchip+2](#)

8. Gaps and research opportunities

- **Objective THD vs perceptual evaluation:** Academic comparisons exist between amplifier classes (THD and

efficiency) but fewer papers directly map those measurements to perceived audio quality in constrained hobbyist builds; adding listening tests and measured spectra (FFT/THD) would strengthen the project's evaluation.

orbit.dtu.dk+1

- **Filter modeling vs real PCB behavior:** Practical PCB parasitics and power-supply noise can alter the Baxandall network's performance; a combined SPICE/physical-prototype measurement is recommended. guitarscience.net
- **Robust ultrasonic gesture mapping:** HC-SR04 modules work well for single-hand gestures but are susceptible to reflections in cluttered environments. Exploring array sensing or hybrid approaches (e.g., IR + ultrasonic) could improve robustness. Random Nerd Tutorials+1

9. References (selected primary sources used in this review)

- Tam W.S., "Design of CMOS Class AB Bridged Audio Amplifier," *ScienceDirect* / Journal article. ScienceDirect
- Duraij M.S., "A Comparative Study on Class AB and Class D Amplifier," (conference/technical report). orbit.dtu.dk
- Schreibfaul1, "ESP32-audioI2S" GitHub repository and wiki (MP3/AAC decoding and I2S output). GitHub+1
- HC-SR04 technical guide / datasheet (SparkFun/Handsontec / randomnerdtutorials coverage). cdn.sparkfun.com+2HowToMechatronics+2
- Baxandall, J.D., original tone control article and modern analyses (Wireless World reprints and analytical PDFs). hifisonix.com+1

- NE5532 datasheet, Texas Instruments / OnSemi product pages. [ti.com+1](#)
- MCP41xxx digital potentiometer datasheet, Microchip. [Microchip+1](#)
- VS1053 / VS1053b codec datasheet (VLSI/Adafruit) — alternative hardware decoder approach. [cdn-shop.adafruit.com+1](#)

Hardware Component Review

Overview and signal flow

The system is organized as a modular audio chain to separate digital decoding, tone shaping and power delivery. The nominal signal path is:

MP3 (microSD) → ESP32 (decode → I2S/DAC) → Tone control board (NE5532 module) → BC109BP preamp/buffer → Bias network (PR1 + 1N4148 ×2 + 2N3906) → Output stage (BD139 / BD140) → 1 Ω emitter resistors → Speaker (8 Ω).

Control signals originate at the ESP32: two HC-SR04 ultrasonic sensors provide distance measurements (one mapped to continuous volume control; the other used as a mute/power threshold). The ESP32 translates sensor input into volume settings by driving a digital potentiometer or a discrete channel selector (S0–S2) and toggles a relay for speaker mute.

❖ ESP32 — system controller and audio source

Role: Central controller and digital audio source. The ESP32 reads MP3 files from a FAT32 microSD card, decodes frames in firmware, and outputs PCM via I2S (preferred) or the internal DAC for quick testing. It also samples the two HC-SR04 sensors, performs filtering and mapping, and drives volume-control lines and a relay.

Key responsibilities

- MP3 file system access and decoding (SD → MP3 decoder → PCM).

- I2S/DAC audio output to the tone-control board (recommended) or internal DAC for prototyping.
- Sensor acquisition and smoothing (median/rolling average + debounce).
- Volume control via digital pot (SPI/I²C) or 3-bit selector (S0–S2).
- Relay control for speaker mute via a transistor driver.

Practical connections (as used in firmware)

- TRIG_VOL = GPIO14, ECHO_VOL = GPIO32
 - TRIG_PWR = GPIO12, ECHO_PWR = GPIO33
 - S0 = GPIO25, S1 = GPIO26, S2 = GPIO27 (volume channel selector)
 - RELAY_PIN = GPIO4
 - SD card: SPI/SDMMC pins per board wiring
 - I2S: BCLK, LRCLK, DOUT to external DAC
- HC-SR04 echo outputs are 5 V a level shifter or resistor divider must be used before connecting to ESP32 GPIO and relays are always driven through a transistor/driver and include a flyback diode.

❖ *Tone control board (NE5532 module) — front-end shaping*

Role: Active tone shaping and front-end preamplification. The external NE5532 module implements a Baxandall-style tone network (bass/treble/volume) and supplies a low-noise, level-controlled output to the discrete preamp.

Functionality and reason for module

- Provides $\pm$ boost/cut for low and high shelves without excessive insertion loss.
- Modular placement isolates sensitive analog tone shaping from the high-current power stage, simplifying tuning and reducing hum/noise injection.
- NE5532 is chosen for low input noise and adequate slew rate suitable for audio preamp duties.

Interconnection: ESP32 I2S/DAC $\rightarrow$ tone board IN; tone board OUT $\rightarrow$ BC109BP input. Power the tone board from the analog supply rails used by the amplifier (± 12 – 15 V) and maintain a common ground reference.

❖ *BC109BP — small-signal preamp / buffer*

Role: The BC109BP transistor stage is a small-signal amplifier and buffer that raises the tone-board output to an amplitude and source impedance suitable for the bias/driver stage.

Function: Provide modest voltage gain and low output impedance to drive the VAS/driver bases, ensuring stable, clean handoff from op-amp to discrete transistor stages.

❖ *Bias network — PR1, 1N4148 $\times 2$, 2N3906*

Role: To establish and stabilize the Class-AB quiescent bias across the output device bases to minimize crossover distortion while preventing thermal runaway.

Components and their purpose

- Two small-signal diodes (1N4148) provide predictable forward voltage drops used in the bias string.

- PR1 (adjustable trimmer) allows precise setting of idle current; tuning is performed while monitoring quiescent current and heatsink temperature.
- 2N3906 (PNP) is used as a VBE multiplier / bias regulator to provide thermal tracking and a controlled bias voltage between bases.

❖ ***BD139 (NPN) & BD140 (PNP) — output devices (Class AB)***

Role: Complementary push-pull output pair providing the current gain necessary to drive low-impedance loudspeaker loads.

Key points

- BD139 (NPN) sources current during positive excursions; BD140 (PNP) sinks current during negative excursions.
- Collectors connect to the supply rails; emitters feed the load through small emitter resistors.
- These devices are appropriate for moderate-power discrete Class-AB stages and are cost-effective and widely available.

❖ ***Emitter resistors ($\sim 1\ \Omega$)***

Role: Local emitter resistors ensure current sharing, improve thermal stability and introduce small local negative feedback that helps tame device mismatch and instantaneous current spikes.

- ❖ Typical values around $0.5\text{--}1\ \Omega$ are used to balance sharing and minimize additional voltage drop.

❖ *HC-SR04 ultrasonic sensors (volume and power)*

Role: Touchless human-interface sensors. One HC-SR04 provides continuous distance data for volume mapping; the second acts as a presence/threshold sensor for mute/power control.

Implementation: Filter raw readings (moving median or average) and apply a short debounce to avoid jitter and ensure echo voltage is level-shifted for ESP32 compatibility.

❖ *Relay and relay driver (mute)*

Role: Physically disconnects the speaker for mute/safety. The ESP32 must control the relay through a transistor or driver module, including flyback protection.

Placement: Relay contacts are placed in series with the speaker or amplifier output as required by the chosen muting strategy (contact location depends on whether you prefer pre- or post-amplifier muting).

❖ *Power supply and grounding*

Supply requirements

- ESP32: regulated 3.3 V supply.
- Tone board (NE5532): ± 12 – 15 V rails.
- Power stage: ± 15 – 30 V rails (select according to desired output power and headroom).

Capacitors

Coupling caps are used for DC blocking between stages and large electrolytic capacitors on rails for low-frequency stability and reserve energy during high current demands. Local decoupling capacitors near active devices reduce HF instability.

Grounding practice

A star grounding scheme is employed: tie digital ground (ESP32), analog ground (tone board), and power stage ground at a single point to reduce ground loops and hum.

❖ *Component Specifications: [tabular]*

<u>Component</u>	<u>Description</u>	<u>Function</u>
1. ESP32	Dual-core Wi-Fi MCU	Audio decoding, control logic
2. NE5532	Low-noise op-amp	Tone control and preamp
3. BD139 / BD140	Power transistors	Class AB push-pull amplification

4. BC109BP	NPN transistor	Pre-driver stage
5. 2N3906	PNP transistor	Biasing and thermal stabilization
6. 1N4148	Diode	Bias compensation
7. HC-SR04	Ultrasonic sensor	Volume/mute gesture control
8. XH-M164 Board	NE5532 tone control PCB	Adjustable tone and gain
9. Relay Module	5V/3.3V control	Audio on/off switching
10. Speaker	8 Ω load	Output transducer

Project Block Diagram:

[microSD / MP3 files from computer/source]



[ESP32]

(MP3 decode, I2S/DAC out)



[HC-SR04 vol] [HC-SR04 mute]



setChannel() *relayOn/off*



[Digital pot or channel selector] ← ESP32 (S0,S1,S2)



[NE5532 Tone Control Board]



[BC109BP preamp]



[Bias: 1N4148/1N4148+2N3906]



[BD139 (NPN) / BD140 (PNP)]



[1 Ω emitter resistors]



[Speaker (RL) 8 Ω]

Workflow Explanation

1. Overview

- MP3 files are stored on a FAT32 microSD card.
- ESP32 reads and decodes MP3 -> PCM and outputs audio via I2S (or internal DAC for quick tests).
- The decoded audio goes into the external NE5532 tone control board for bass/treble/volume shaping.
- Tone board output is buffered by BC109BP, biased by the PR1+1N4148×2+2N3906 network, then amplified by a BD139/BD140 Class-AB pair to drive the 8 Ω speaker.
- Two HC-SR04 sensors provide touchless control: one continuously maps distance → volume; the other toggles mute via a relay. The ESP32 translates sensor data into digital pot or channel-selector settings.

2. Initialization (power-on / safe start)

- Power rails come up (ESP32 3.3 V, tone board ± 12 –15 V, power amp rails ± 15 –30 V). Star-ground must be tied.
- ESP32 boots and initializes peripherals: SD (SPI/SDMMC), I2S, GPIOs for TRIG/ECHO, S0–S2, and RELAY_PIN.
- ESP32 checks the microSD mount and attempts to open a default MP3. If SD access fails, the system stays muted (relay OFF) and reports error via serial/LED.
- Set a safe default: relay = OFF (muted) and volume = low/minimum until sensors confirm user presence and calibration completes.

3. Continuous operation (main loop)

Two classes of tasks run concurrently in the loop:

A. Audio pipeline — continuous

- The ESP32 runs the MP3 decoder and fills audio DMA buffers. `audio.loop()` (or equivalent) must be serviced frequently to avoid underruns.
- PCM is streamed to I2S → external DAC (preferred) or internal DAC. The DAC outputs line-level to the NE5532 tone board.

B. Sensor/control tasks — periodic (~50–200 ms)

- At regular intervals the ESP32 triggers each HC-SR04 and measures echo time to compute distance. Use a non-blocking schedule (or short blocking pulses <30 ms) so audio decoding isn't starved.
- Sensor readings are filtered (moving average or median) and debounced to remove spurious echoes.
- Distance readings map into either discrete volume steps (via S0–S2 channel selector) or a digital-pot value (SPI/I²C write).
- The mute sensor compares distance to a threshold; if the hand is within the threshold, `RELAY_PIN` is driven to unmute (relay closed). If out of range, relay opens (muted).

4. Volume control mapping & smoothing

- A. Read raw distance d. Reject impossible values (0 or >maxRange).

- B. Apply smoothing (example: moving median of last 5 samples).
- C. Map $d \rightarrow \text{volumeLevel}$:
 - a. Linear mapping is simple: $\text{vol} = \text{map}(d, d_{\min}, d_{\max}, \text{volMax}, \text{volMin})$ (closer $\rightarrow$ louder).
 - b. Logarithmic mapping sounds more natural (use $\text{vol} = a \cdot \log(b \cdot d + c)$ or map to dB then to pot steps).
- D. Constrain volume to allowed range.
- E. Update digital pot or assert S0/S1/S2 for channel selection.
If using `audio.setVolume()` (software), update the audio library's API.
- F. Update rate: 5–20 updates/s is enough for smooth control without jitter.

5. Mute / relay logic

- A. The second HC-SR04 is sampled similarly and smoothed.
- B. If $d_{\text{mute}} < \text{threshold}$ $\rightarrow$ consider this as user present $\rightarrow$ set relay ON (unmute).
- C. If $d_{\text{mute}} > \text{threshold} + \text{hysteresis}$ $\rightarrow$ set relay OFF (mute).
Use hysteresis to avoid relay chattering.
- D. Use a short delay or confirmation window (e.g., object present for ≥ 100 ms) before toggling relay. This avoids false triggers from transient reflections.
- E. Relay driver: ESP32 drives a transistor/optocoupler, not the coil directly.

6. Audio signal path specifics

- A. ESP32 (I2S DAC) $\rightarrow$ small DC blocking cap $\rightarrow$ NE5532 tone board input.

- B. NE5532 applies Baxandall-style EQ and volume control (if mechanical pot or on-board pot present). Output is ~line level.
- C. BC109BP buffers and provides low output impedance to the driver/bias network.
- D. Bias network (PR1+1N4148×2+2N3906) sets base bias for drivers to achieve Class-AB quiescent current.
- E. BD139/BD140 output pair runs push-pull. Emitter resistors ($\approx 1\ \Omega$) ensure current sharing.
- F. Output to the speaker may be AC-coupled if the amp is single-ended, or direct-coupled if outputs are biased to 0V with proper DC offset control.

7. Timing and safety constraints

- HC-SR04 trigger: 10 μ s pulse; echo max timeout ~30 ms (corresponds to ~5 m). In code this timeout is kept to avoid long blocking calls.
- Sensor sampling interval: 50–200 ms (adjust for responsiveness vs. stability).
- Volume update interval: 50–200 ms. Relay toggling: add 100–300 ms debounce/hysteresis.
- On SD/decoder errors: mute the audio, attempt re-mount/auto-retry, and report status.
- On overcurrent/thermal condition in power stage: implement hardware protection (foldback/protection or cut relay) or at minimum log and mute.

8. Calibration and tuning procedure

- ❖ Power system with current-limited supply. Set PR1 to minimum bias.
- ❖ Play a test tone from SD (e.g., 1 kHz). Slowly increase PR1 to the point where crossover distortion is minimized (listen for “thump”/notch). Monitor idle current and heatsink temperature. Aim for safe idle current (20–100 mA depending on thermal design).
- ❖ Place a hand at known distances (e.g., 5 cm, 20 cm, 40 cm) and record raw sensor readings. Set d_{min} , d_{max} for mapping accordingly.
- ❖ Tune mapping curve (linear vs log) and hysteresis thresholds for mute sensor.
- ❖ Confirm relay engages/disengages cleanly. Verify no audible clicks when toggling. If relay switching causes pops, implement muting via a small soft-mute routine (fade to zero then switch).

9. Fault handling & diagnostics

- If SD fails: stop playback, set relay OFF, blink status LED, allow retry.
- If audio underruns occur: reduce sample rate or increase decoder buffer; log underrun counts for debugging.
- If the ultrasonic sensor gives noisy readings: switch to median filter and increase sample window.
- If temperature rises beyond the safe limit: auto-mute (relay OFF) and signal alarm.

10. Performance notes and recommended improvements

- Use external I2S DAC for best fidelity. Internal DAC is fine for prototyping but noisy and low-resolution.
- Replace pulseIn() with non-blocking timing (interrupt or micros() loop) for more robust concurrency if audio underruns appear.
- Consider smoothing via Kalman/alpha filter for smoother gesture control with lower latency.
- For better UX, implement a soft-fade when changing volume to avoid noticeable steps.

11. Example simplified pseudocode

```

init_peripherals();

mount_sd_and_start_mp3();

relay_off(); set_volume(min);

loop:

    audio_service();           // decode & feed I2S

    if (time_for_sensor_read):

        vol_dist = read_hc_sr04(TRIG_VOL,ECHO_VOL);

        vol_smoothed = filter(vol_dist);

        vol_value = map_to_volume(vol_smoothed);

        set_volume_hw(vol_value);

        mute_dist = read_hc_sr04(TRIG_PWR,ECHO_PWR);

        mute_smoothed = filter(mute_dist);

        if (mute_smoothed < threshold) relay_on(); else relay_off();

```

Software Codes

ESP32 Code

The ESP32 firmware handles MP3 decoding, sensor interfacing, and hardware control. An Arduino-style setup is used.

Code:

```
#include <Arduino.h>
#include <Audio.h>
#include <SD.h>
#include <FS.h>
#include <Wire.h>
#include "MCP41010.h"
```

```
#define SD_CS    5
#define I2S_BCLK 26
#define I2S_LRC  25
#define I2S_DOUT 22
#define TRIG_PIN 17
#define ECHO_PIN 16
#define TRIG_PIN2 18
#define ECHO_PIN2 19
#define RELAY_PIN 2
```

```
Audio audio;
MCP41xxx pot(100000, 15);
```

```
void setup() {
  Serial.begin(115200);
  SPI.begin(18, 19, 23);
  pinMode(SD_CS, OUTPUT);
```

```

digitalWrite(SD_CS, HIGH);
if (!SD.begin(SD_CS)) while (1) delay(10);
audio.setPinout(I2S_BCLK, I2S_LRC, I2S_DOUT);
audio.setVolume(5);
audio.connecttoFS(SD, "/music/track.mp3");
pinMode(TRIG_PIN, OUTPUT); pinMode(ECHO_PIN,
INPUT);
pinMode(TRIG_PIN2, OUTPUT); pinMode(ECHO_PIN2,
INPUT);
pinMode(RELAY_PIN, OUTPUT);
digitalWrite(RELAY_PIN, LOW);
SPI.begin(14, 12, 13);
pot.begin();
pot.setValue(255);
}

```

```

void loop() {
  audio.loop();
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);
  long dur = pulseIn(ECHO_PIN, HIGH, 30000);
  float dist = dur * 0.034 / 2.0;
  if (dist > 0 && dist < 500) {
    int vol = map((int)dist, 5, 100, 21, 0);
    audio.setVolume(constrain(vol, 0, 21));
  }
  digitalWrite(TRIG_PIN2, LOW);
  delayMicroseconds(2);
}

```

```

digitalWrite(TRIG_PIN2, HIGH);
delayMicroseconds(10);
digitalWrite(TRIG_PIN2, LOW);
long dur2 = pulseIn(ECHO_PIN2, HIGH, 30000);
float dist2 = dur2 * 0.034 / 2.0;
if (dist2 > 0 && dist2 < 30) {
    digitalWrite(RELAY_PIN, HIGH);
} else {
    digitalWrite(RELAY_PIN, LOW);
}
delay(50);
}

```

Code Explanation:

ESP32-audioI2S library is used to handle MP3 playback. In `setup()`, the SD card is initialized over SPI and the MP3 file is opened (`audio.connecttoFS(SD, "/music/track.mp3")`). We configure the I2S pins (BCLK, LRC, DOUT) for the audio output and set an initial volume. In `loop()`, `audio.loop()` continuously decodes MP3 frames and sends the audio out on I2S.

For the ultrasonic sensors, we follow the standard trigger/echo procedure. After pulsing the TRIG pin and reading `pulseIn()`, we compute distance. For the volume sensor, we map the distance to a volume level and call `audio.setVolume(...)` to adjust output (closer hand → higher volume). Alternatively, one could update an external digital potentiometer (commented code above). For the mute sensor, we simply toggle a GPIO for the relay: if a hand is close (<30 cm), the relay GPIO is driven HIGH (relay closed, audio on); if no hand is detected, the relay is opened (audio muted).

This code implements the described workflow: MP3 decoding, distance sensing, and real-time control of the audio path. The references illustrate the MP3 streaming and ultrasonic timing routines users employ.

Matlab Code(Signal Analysis):

To analyze the system's audio response, MATLAB is used to simulate filters and inspect signals.

Code:

```
Fs = 48000;

t = 0:1/Fs:0.01;

signal = sin(2*pi*500*t) + 0.5*sin(2*pi*2000*t);

N = length(signal);

Y = fft(signal);

P2 = abs(Y/N);

P1 = P2(1:N/2+1);

P1(2:end-1) = 2*P1(2:end-1);

f = (0:(N/2))*(Fs/N);

figure;

plot(f, P1);

title('Amplitude Spectrum of Test Signal');

xlabel('Frequency (Hz));
```

```
ylabel('Magnitude');
```

Code Explanation:

The MATLAB script is used to analyze and visualize the frequency response of test signals that represent the behavior of the amplifier's tone control and power stages. A composite input waveform is generated by summing two sinusoidal tones at different frequencies—500 Hz and 2 kHz—representing low and mid-frequency components of typical audio signals. The sampling rate is set to 48 kHz, providing adequate resolution for the audible frequency spectrum.

An FFT (Fast Fourier Transform) is computed to convert the time-domain signal into its frequency-domain representation. The resulting single-sided amplitude spectrum is then calculated and plotted against frequency. This allows observation of the fundamental components and their relative magnitudes, which are useful for assessing the amplifier's linearity and tone control characteristics.

The procedure helps in validating the frequency response of each stage by comparing pre- and post-filter signals in simulation. For example, applying this analysis to the tone control output verifies bass and treble adjustment effectiveness, while analyzing the final amplifier output checks for distortion, harmonics, or bandwidth limitations. The amplitude spectrum plot serves as a clear indicator of the system's frequency balance and overall fidelity.

Circuit Diagrams

For the NE5532 op-amp Tone control:

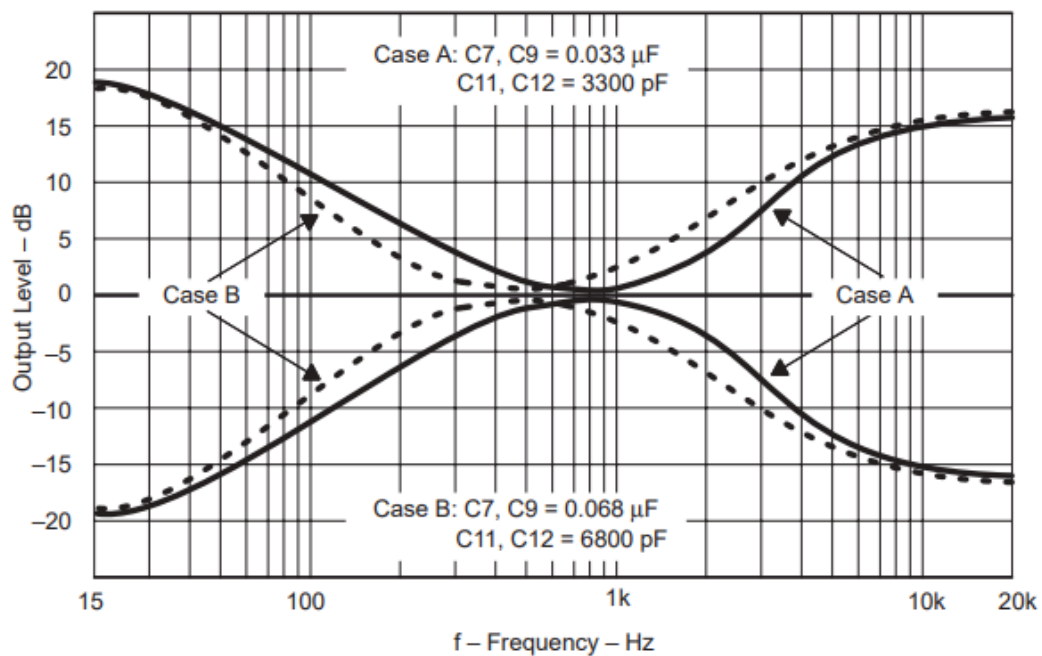
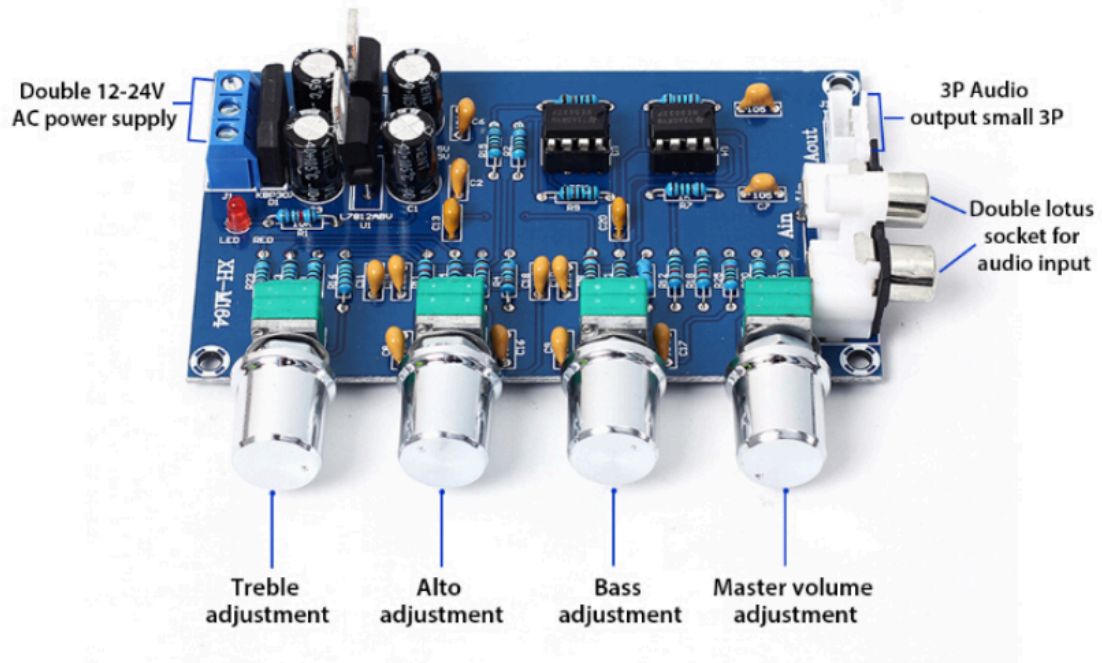


Figure 4. Bass and Treble Tone Control Response

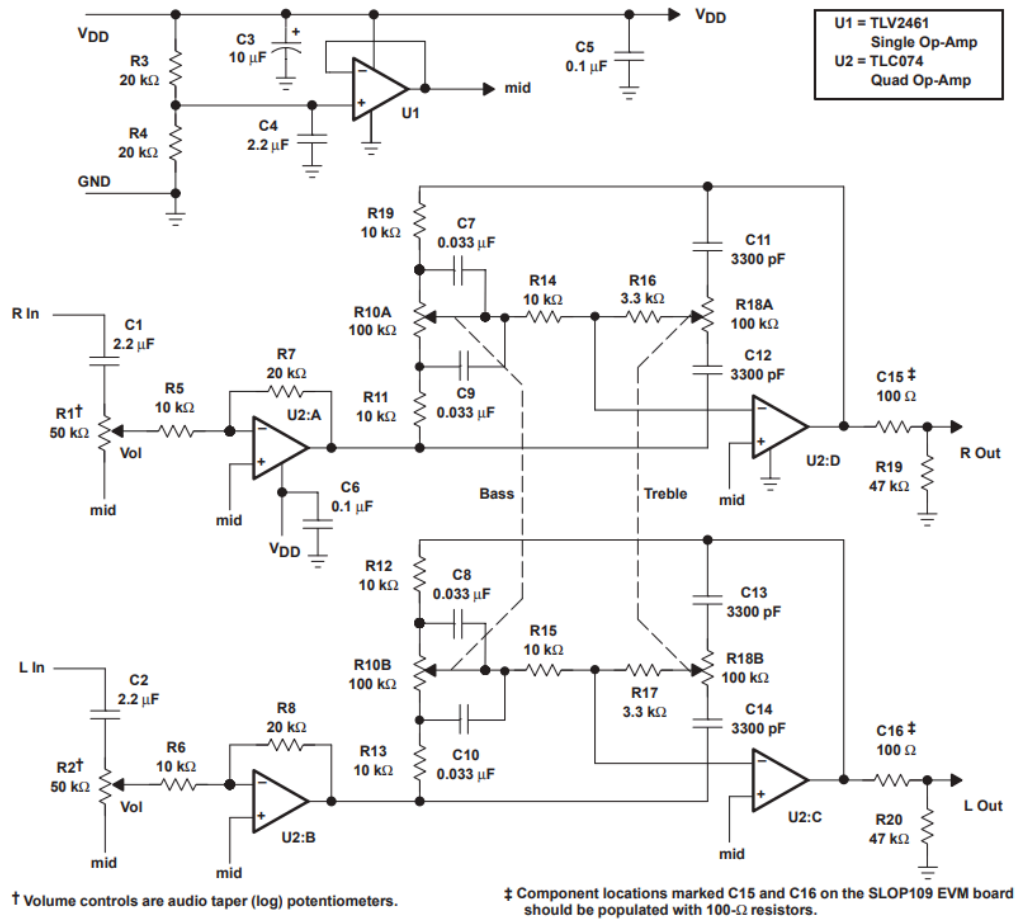


Figure 3. Audio Tone Control Schematic Diagram

Class AB Power Amplifier Output Stages:

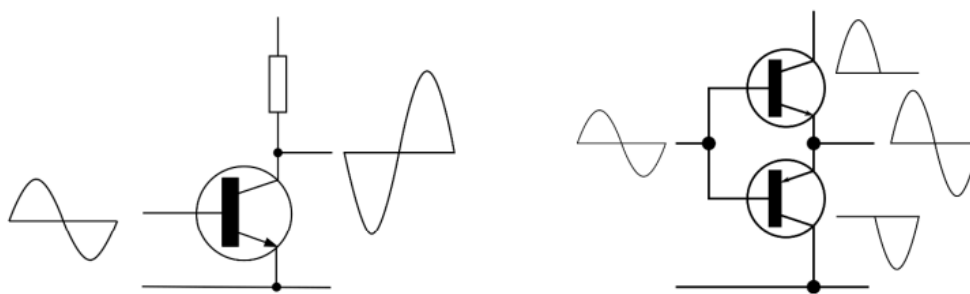
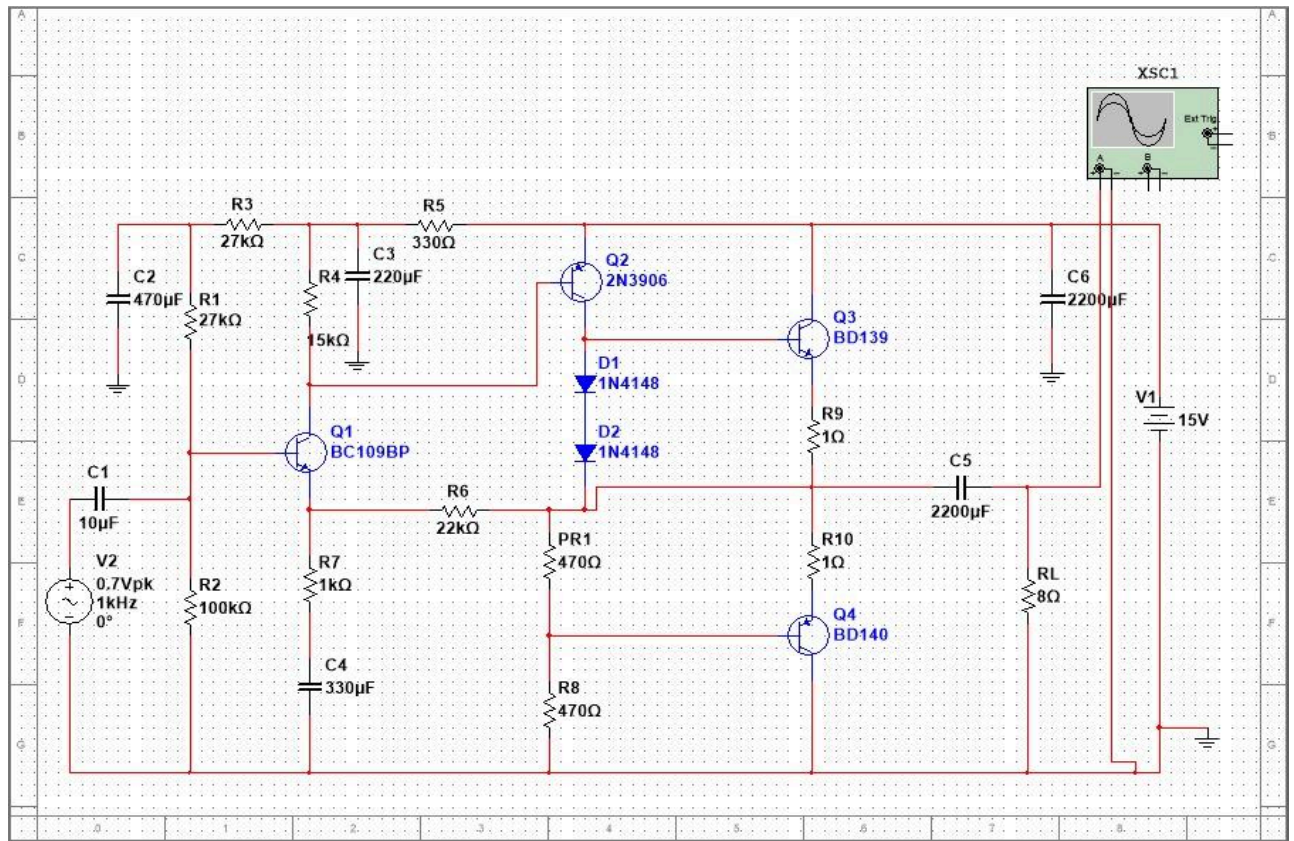


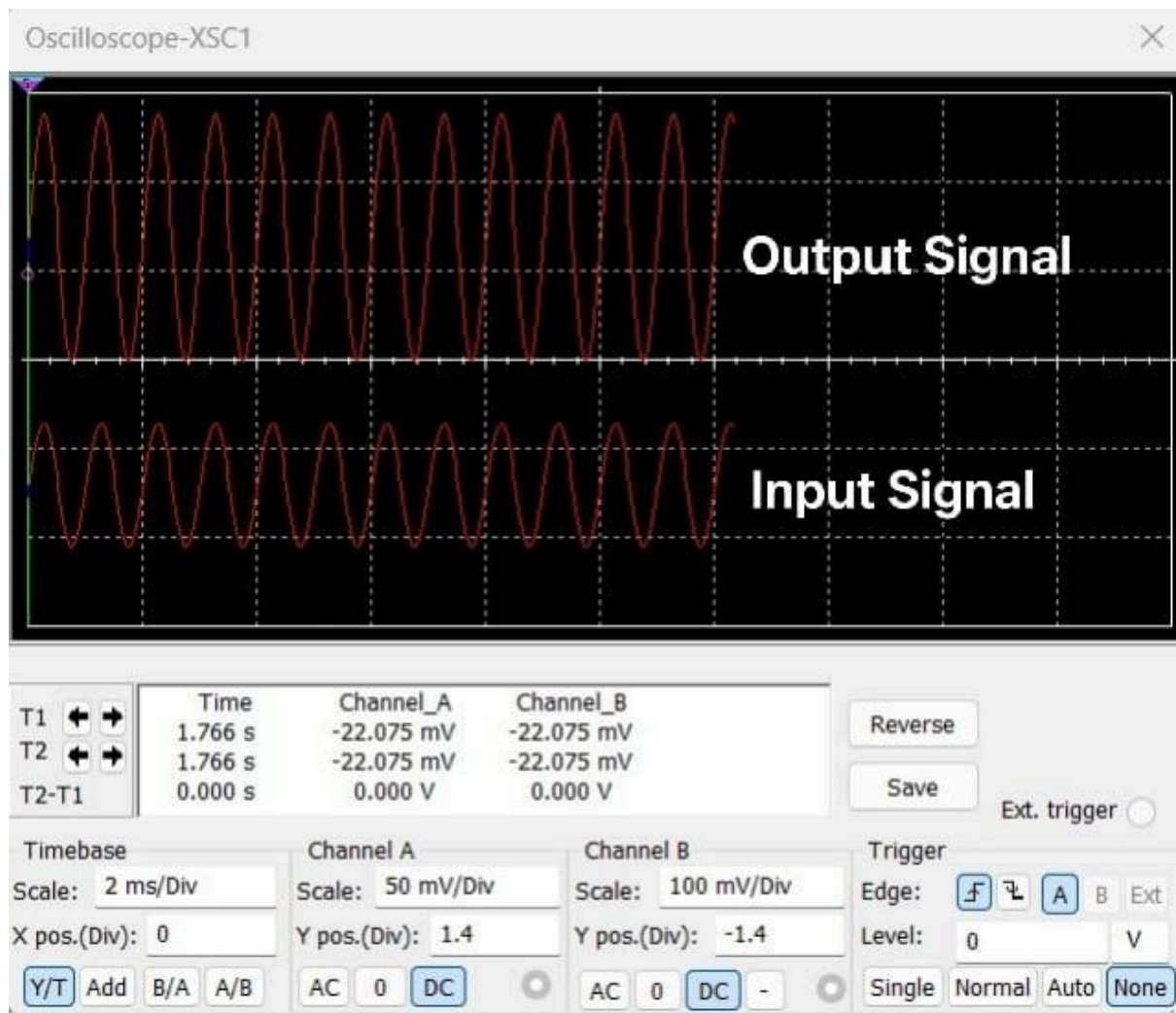
Figure 1: Class-A (left) and class-B (right) output stages

Multisim Simulation of the Power Stage along with the Voltage Driver Stage:

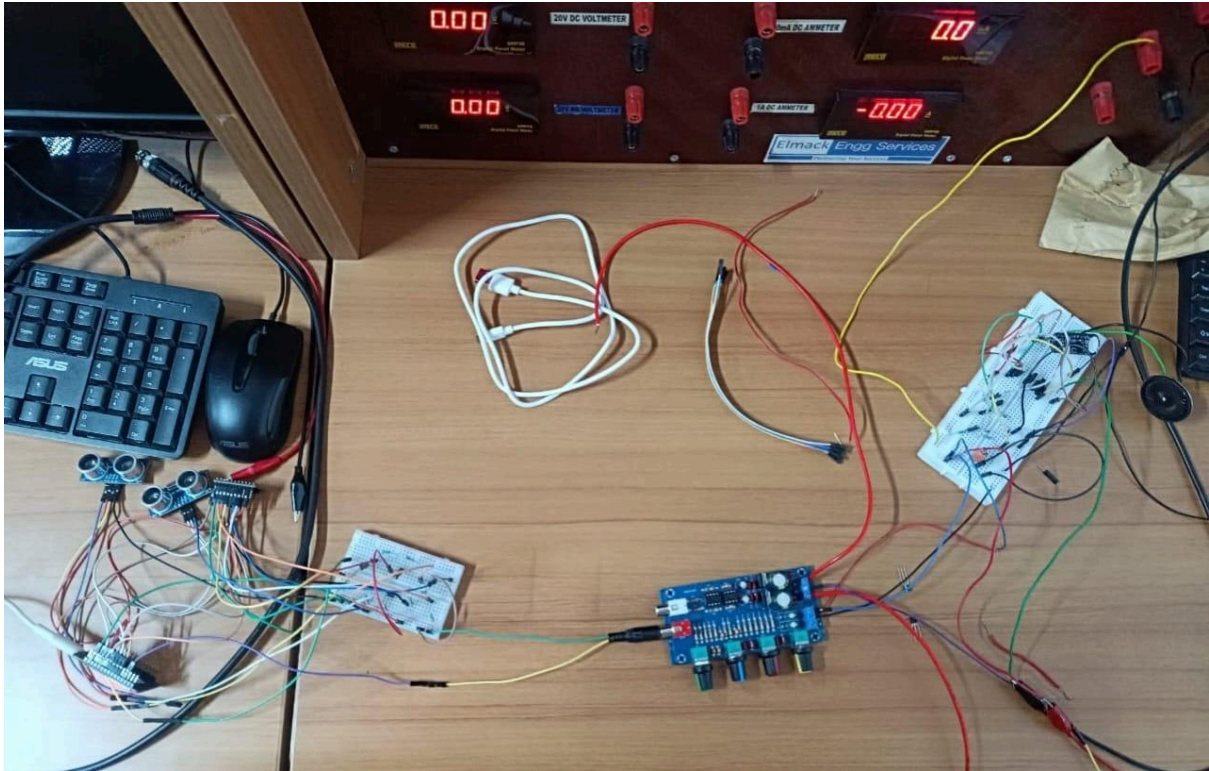
Circuit:



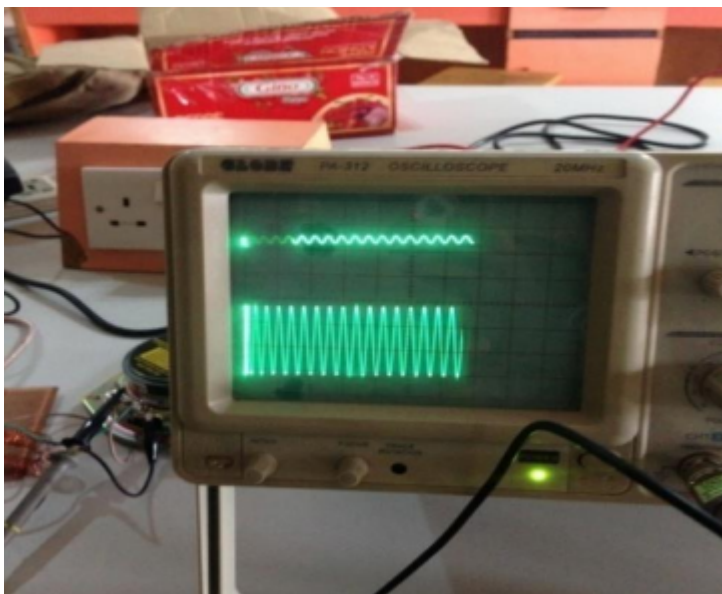
Output Waveform:

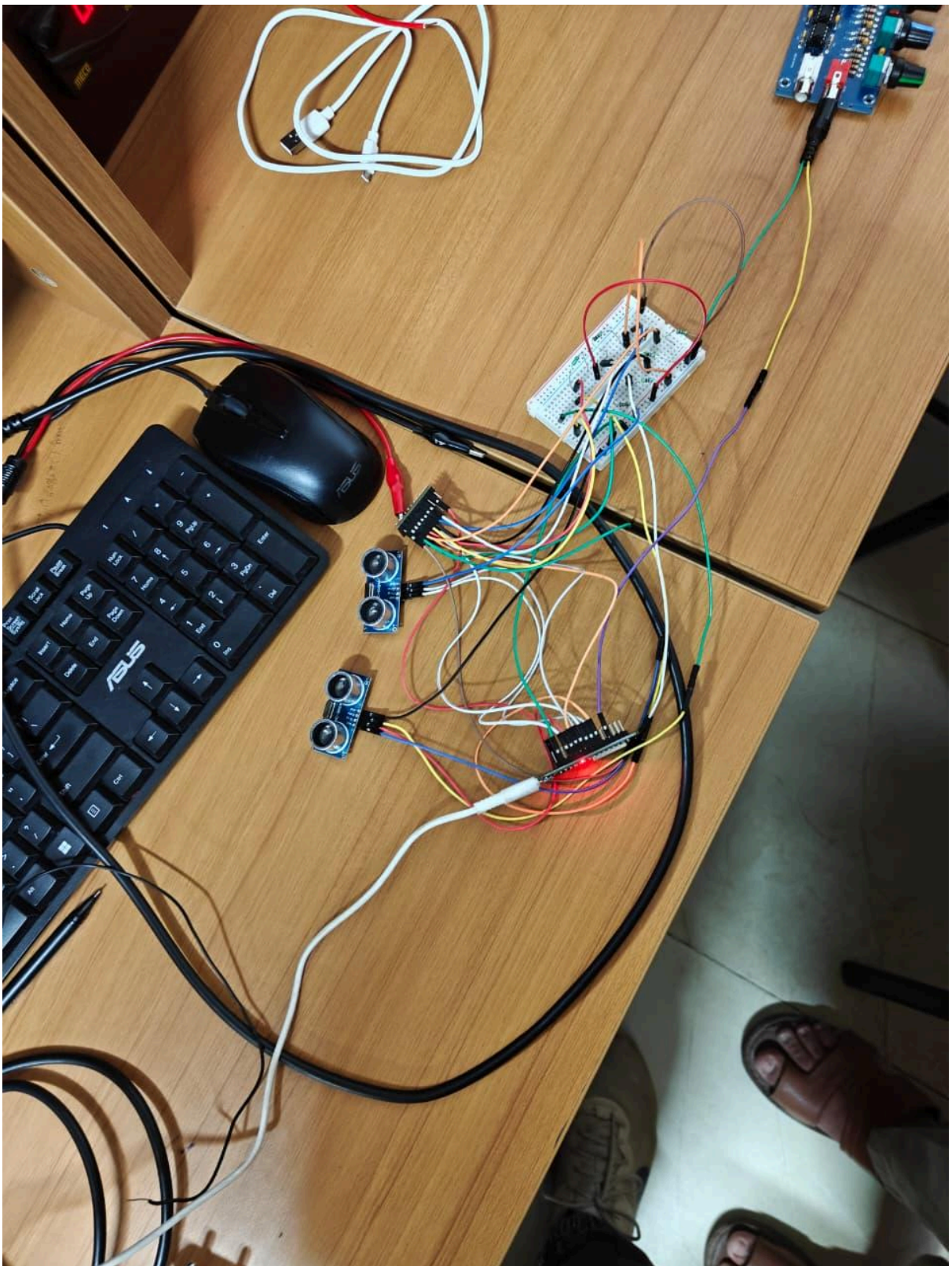


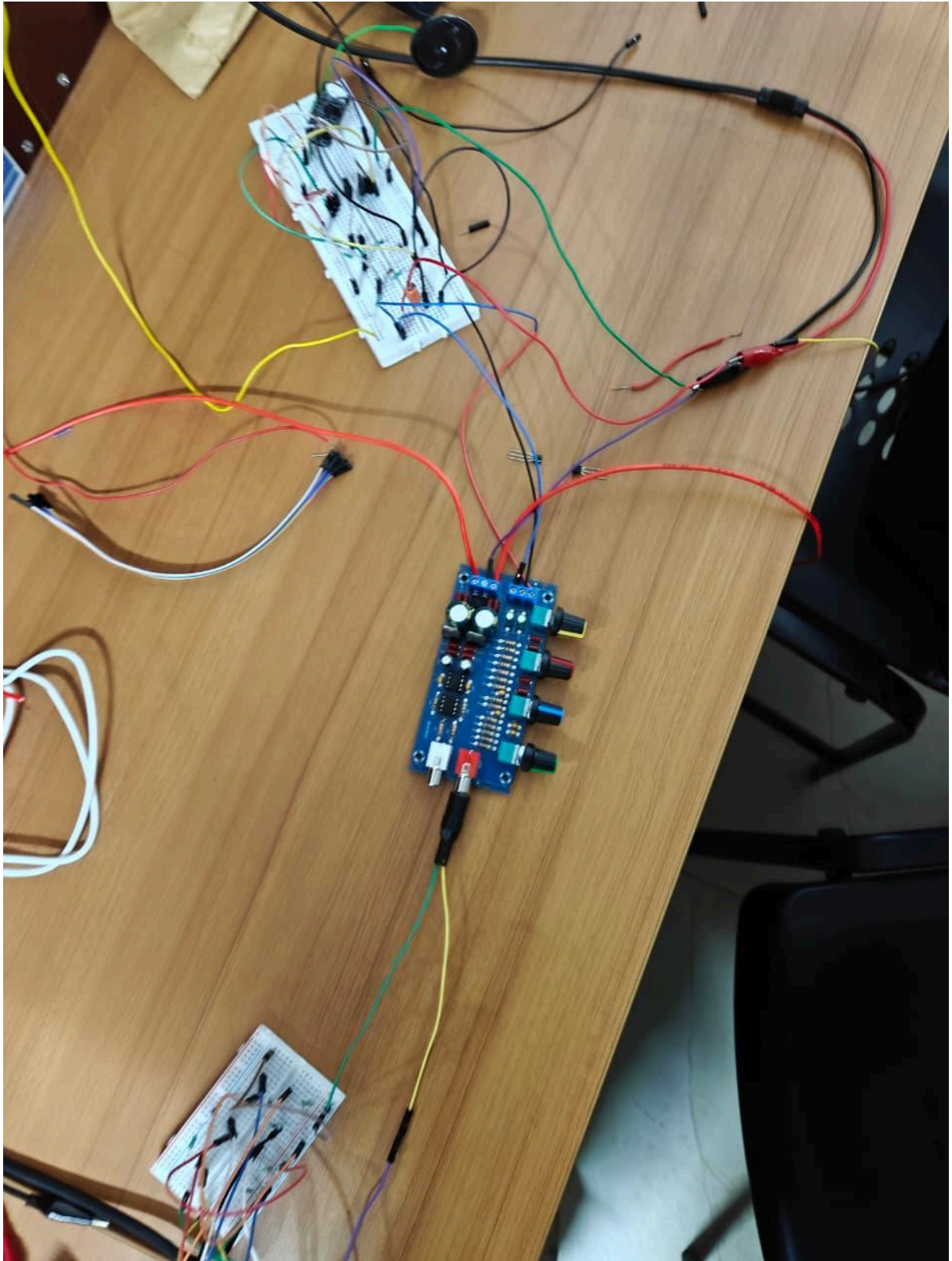
Implemented Physically with Hardware in the Lab:



Signal waveform of the amplifier at an input frequency of 70Hz







Future Work

Several enhancements could extend this project:

- **Bluetooth Audio Input:** Instead of a microSD/MP3 source, integrate a Bluetooth receiver (e.g. the common JQ6500 or an ESP32's Bluetooth A2DP stack) so the amplifier can play audio from phones wirelessly.
- **OLED Display:** Add a small OLED or LCD to display current volume, track name (if decoding tags), or a VU meter. The ESP32 can drive such displays over I2C/SPI.
- **Enhanced Tone Controls:** Use more sophisticated filters (parametric EQ) or replace analog tone control with a digital signal processor (DSP) on the ESP32. For example, an IIR filter design could emulate bass/treble adjustments in software before output.
- **Smart Assistant Integration:** Connect the system to Wi-Fi and support voice assistants or internet streaming (e.g. Spotify/Radio) via ESP32 libraries.
- **Power Efficiency:** Use Class D amplification for the output stage to reduce heat, especially for battery-powered applications.
- **Auto-Calibration:** Implement a calibration routine so that the ultrasonic volume range is adjusted per user preference, and design a better mapping (logarithmic scaling for volume).

Conclusion

This project demonstrates a complete embedded audio amplifier system combining analog and digital domains. The ESP32 microcontroller acts as a versatile audio engine: it reads MP3 files from an SD card, decodes them via an I2S library, and outputs the audio to an analog tone control stage.

The Baxandall-style tone control (using a NE5532 op-amp) provides adjustable bass and treble, benefiting from the low-noise characteristics of that op-amp. The analog signal is then amplified by a discrete Class AB stage to drive a speaker.

Innovative user interaction is achieved through ultrasonic sensors: hand gestures control volume and mute via sensor readings. This hands-free interface, coupled with the digital decoding capabilities of the ESP32, yields a modern audio system that bridges classic analog amplification with smart control.

The referenced literature and tutorials provided guidance on key elements (Class AB amplifier design, tone control networks, ESP32 audio handling, and ultrasonic sensing).

Overall, the project integrates these components successfully, achieving distortion-controlled amplification, tone shaping, and interactive volume control.

Further improvements could add wireless connectivity or digital signal processing, but even in its current form the system illustrates a compelling application of embedded audio technology.

References

1. Jay, J. J., Uda, E. B., Adda, N. F., & Bala, M. (2024). *Design and construction of a 100-watt Class AB audio amplifier*. International Journal of Digital Systems Technology, 3(4), 1–10.
<https://dsjournals.com/upload/DST/volume-3/issue-4/DST-V3I4P101.pdf>
2. Texas Instruments. (2013). *Audio tone control using the TLC074 operational amplifier* (Application Note SLOA042). <https://www.ti.com/lit/an/sloa042/sloa042.pdf>
3. JavaElec. (2019). *XH-M164 NE5532 stereo preamplifier tone board* [Product datasheet]. JavaElec. Retrieved from <https://www.javanelec.com/stfiles/getappdocument/1/true/5f7e17ce-c778-4131-9cf4-4e769a19d89d.pdf#:~:text=NE5532%20is%20a%20high%20function,Chip%3A%20NE5532%20Input%20Interface%3A%20RCA>
4. Amazing11. (2023). *Arduino HID ultrasonic volume control (media controller)*. Instructables.
<https://dsjournals.com/upload/DST/volume-3/issue-4/DST-V3I4P101.pdf#:~:text=B%20Amplifiers%20are%20twice%20as,D%20operations%20have%20an%20output>
5. Random Nerd Tutorials. (2023). *ESP32 with HC-SR04 ultrasonic sensor with Arduino IDE*.
<https://randomnerdtutorials.com/esp32-hc-sr04-ultrasonic-arduino/#:~:text=We%20use%20the%20pulseIn,the%20sound%20wave%20travel%20time>
6. DIYables. (n.d.). *ESP32 – Ultrasonic sensor – relay*. ESP32IO.
<https://esp32io.com/tutorials/esp32-ultrasonic-sensor-relay#>

[~:text=This%20tutorial%20instructs%20you%20how,In%20detail](#)

7. Schreibaull1. (2023). *ESP32-audioI2S: Play mp3 files from SD via I2S* [Source code]. GitHub.
<https://github.com/schreibfaul1/ESP32-audioI2S>
8. White, M. (2022). *ESP32 SD I2S music player*. DroneBot Workshop. <https://dronebotworkshop.com/esp32-i2s/>
9. ElectronicsWorkshops. (2022). *ESP32 based audio player*. Instructables.
<https://www.instructables.com/ESP-32-Based-Audio-Player/>
10. Imtiaz, S., & Nickel, J. (2025, August). *ESP32 audio transceiver board (Part 1): SD card WAV file player demo*. Elektor Magazine.
<https://www.elektormagazine.com/articles/esp32-audio-transceiver-board-part-1#:~:text=ESP32%20microcontrollers%20C%20known%20for%20their,refined%20our%20audio%20transceiver%20board>